

Whop pixel install: Elyxion Research

Prepared for Elyxion Research, 2026-09-04. Account id `biz_S8kaG0ddZZC4UD`.

One page, for the developer doing the install. Written 2026-08-31. Source: Whop's own pixel guide, <https://docs.whop.com/developer/guides/pixel>. The snippet below was extracted from that page programmatically, not retyped.

Who this is for: a store that runs on its own site (Shopify, Vercel, WordPress, Webflow, Framer) rather than on Whop checkout. A store that sells through Whop checkout needs step 1 only; Whop records checkout views, purchases, subscriptions and trials server-side with no extra code.

Before anything else, check the account id in step 1. It has to be the Whop business id for this specific store, which was sent to you alongside this document. A placeholder left unreplaced is the single most common reason a pixel reports nothing at all, and it fails silently: the snippet loads, the page looks fine, and no data ever arrives.

Step 1. The base snippet, on EVERY page

Paste inside `<head>` on every page in the funnel: landing pages, advertorials, quizzes, checkout, and the thank-you / order-confirmation page. **Not just the homepage.** If the platform has a "custom header code" or "site-wide `<head>` code" field, put it there so it ships on every template automatically.

Set the account id to the Whop `biz_` id for this store. In this copy it is `biz_S8kaG0ddZZC4UD`.

```
<script>
!function(w,d,s,u,n,a,b){if(w[n])return;a=w[n]={q:[],t:+new Date,s:[],o:u,track:function(){a.q.push([+new Date].concat([].slice.call(arguments)))},setScope:function(){a.s=[].slice.call(arguments).filter(function(x){return typeof x=="string"});a.q.push([+new Date,"setScope"].concat(a.s))},scope:function(){var c=[].slice.call(arguments);return{track:function(){a.q.push([+new Date].concat([].slice.call(arguments)).concat([{"__scope":c}])}}}};b=d.createElement(s);b.async=1;b.src=u+"/s.js";d.getElementsByTagName(s)[0].parentNode.insertBefore(b,d.getElementsByTagName(s)[0])}(window,document,"script","https://t.whop.tw","whop");

whop.setScope("biz_S8kaG0ddZZC4UD");
whop.track("page");
</script>
```

The thank-you page is the one people forget. Conversion events usually fire there, and if the snippet is missing from it, the purchase is invisible no matter how good the rest of the install is.

Step 2. Fire the purchase event, for off-Whop stores only

Whop cannot see a sale that happens on the store's own rails, so the site has to report it. On the order-confirmation page, after the order succeeds:

```
whop.track("whop_purchase", {
  value: 129.00,           // order total as a NUMBER, not a string, not "$129"
  currency: "USD",
  event_id: "ORDER-10482" // the store's own order number. See step 3.
});
```

THE EVENT NAME IS `whop_purchase`, ALL LOWERCASE, EXACTLY. It has to match the custom conversion the ad campaigns optimize toward. A different name, or a capital letter, produces a separate event that the campaigns cannot see and that reports nothing. A phone's autocorrect has mangled an event name this way before, so paste it, do not type it.

If the order confirmation is a redirect to a new page, fire it on the page they land on. If the checkout completes in place with no redirect, fire it in the success handler after the call resolves.

>>> FIRST CHECK THAT STEP 1 ACTUALLY REACHED THE PAGE YOU ARE ABOUT TO PUT THIS ON.
<<< This has now been the real fault on two client stores in a row, and in both the site looked correctly instrumented. The purchase page is very often the one page step 1 never reached:

- the checkout and thank-you pages are served by a **different platform** than the store (Tagada, Shopify checkout, a hosted payment page), so a snippet added to "the site" is not on them at all; or
- the app has a **hardcoded path allowlist** deciding where analytics load, and the post-payment path is not in it.

The failure is silent in the worst way. A purchase block that begins `if (typeof whop === "undefined") return;` does exactly nothing, forever, and reports no error. Every other event keeps arriving, so the dashboard looks alive while **no sale is ever attributed**.

Verify, do not assume: open the real post-payment page in a browser and evaluate `typeof window.whop`. It must be `"object"`. Use a page you know is working as a positive control in the same session. Do not grep the source for this - pixels are often injected at runtime and a source grep proves nothing either way.

If step 1 cannot reach that page, put the base snippet inside the purchase block itself so the block is self contained and does not depend on a second change landing correctly somewhere else. That is what made it work on the two stores above.

Step 3. `event_id` is not optional here, it is what stops double counting

Whop counts each `event_name` + `event_id` pair **once**. Use the real order number.

- **Correct:** a different `event_id` per order (`ORDER-10482`, `ORDER-10483`, ...).
- **Wrong:** the same literal string for every order. Whop then treats every sale as one repeat of a single event, and hundreds of orders report as one.
- **Wrong:** a value invented at fire time (a timestamp, a random id). It is new on every page load, so a customer refreshing the thank-you page counts as a second sale.

If this conversion is ever also sent from the **server**, it must carry the **same** `event_id` as the browser sent for that order. That is the only thing preventing every sale from being counted twice and the revenue figure doubling.

Step 4. Optional, improves match rates

If the site already has the customer's details at the moment the event fires, attach them:

```
whop.track("whop_purchase", {  
  value: 129.00, currency: "USD", event_id: "ORDER-10482",  
  email: "customer@example.com", first_name: "Jane", last_name: "Doe",  
  phone: "+15551234567", external_id: "cust_123"  
});
```

Send these as plain text. A pre-hashed email is dropped because it is not a valid address, and a hashed phone number is stored as meaningless digits that match nobody. Send only the fields the site actually has.

Step 5. Other funnel steps worth tracking

Standard names, each optionally taking `value` and `currency`: `lead`, `schedule`, `submit_application`, `contact`, `complete_registration`, `view_content`, `add_to_cart`. Any other name is a custom event.

Keep custom event names under about 34 characters. Whop stores longer names but only forwards names under that length to Meta as custom conversions, and a name Meta never receives cannot be optimized toward.

How to verify the install, before any traffic

Do not take "it's installed" on trust. Two ways to confirm, both used before a campaign launches:

1. `POST /events/validate_pixel` with this store's `account_id` and the exact destination URL. `installed` is true only when the pixel was actually seen. Needs an API key **generated inside this store's own Whop account** (a key is scoped to one company, so a key issued by any other account will not work).
2. This store's Whop dashboard, Events view, filtered to that hostname. A zero only means something if a control hostname returns rows, otherwise you are reading a filter you have not proven works.

Whop's own campaign builder also blocks launch when the ad group's conversion event is not live on the destination URL, so a bad install surfaces at the worst possible moment: after the whole campaign is built. Check it first.

What this install does NOT do

It will not populate Whop's built-in **Return** and **ROAS** columns. Those count Whop checkout payments only, and an off-Whop store has none, so they stay at zero no matter how correct this install is. Revenue lands in the custom conversion instead, which is what the **Custom ROAS** metric reads. This is a property of the platform, not a defect in the install, and no amount of developer work changes it. It was confirmed directly with Whop support.

Platform differences: less than people expect

Added 2026-08-31, on the question of whether a custom build installs differently from WordPress.

The code is identical on every platform. What changes is where it goes and who has to do it.

	Base snippet (step 1)	Purchase event (step 2)
WordPress / Shopify / Webflow / Framer	Site-wide header code field or a header-snippets plugin. Non-technical, a paste.	Still a developer. On WooCommerce it is a hook on the order-received page to inject the real total and order number.
Custom build (Next.js, React, Vercel)	Root layout or main template, so it loads on every route. Small job for whoever built the site.	Wherever the order confirmation is rendered, reading the real order object.

The hard part is the same everywhere, and it is not the platform. The site has to hand the tracking call the **real order number and the real order total at the moment the sale completes**. That is a developer task on WordPress exactly as much as on a custom build. Anyone budgeting this as "just paste a snippet" has budgeted for step 1 and forgotten step 2.

The one genuine custom-build trap: client-side routing

Read the snippet in step 1 and note that `whop.track("page")` is a **single top-level call**. It runs once, when the script executes.

- On a **server-rendered site** every navigation is a fresh page load, so the snippet re-runs and each page view fires. Nothing to do.
- On a **single-page app** the script loads once and client-side route changes never re-run it, so **only the first page of the visit is ever recorded**. The developer has to call `whop.track("page")` on route change.

Confidence, stated rather than implied: Whop's pixel guide says nothing about SPAs or routers, so this is not a documented Whop behavior. It is read directly off the snippet's own code, where the call is plainly one-shot with no router hook in the loader. Treat it as very likely and confirm it on the first single-page app it is installed on, rather than quoting it as vendor guidance.

How much does it matter? Less than it sounds. Page views feed attribution and audience quality, but the load-bearing event is the purchase, and that fires correctly as long as the confirmation page calls it. The visible symptom is page-view counts that look too low, which is worth flagging up front rather than explaining later.

If this store is built on Base44

Added 2026-09-04, answering the specific question. Every fact here is from Base44's own documentation, not inferred from how similar builders behave.

Short answer: yes, the pixel has to be installed directly. Whop has no Base44 app or native integration, so nothing installs it for you. The good news is that Base44 supports exactly this.

Where the base snippet goes

Base44's documentation states that you can use the **Code tab** in the app dashboard to edit the generated `index.html`, and it names this as the place for *"trusted script tags and meta tags that need to appear directly in the HTML source, such as analytics, advertising, verification, or other third-party scripts."*

Put the step 1 snippet there, in the `<head>`. That is one paste and it covers every route, because `index.html` is the shell the whole app loads from.

Older third-party write-ups say Base44 does not expose the document head and tell you to append a script from a `useEffect` in `Layout.js` instead. **That workaround is no longer necessary**; the vendor's current documentation describes direct `index.html` editing. If the Code tab is unavailable on your plan, the `Layout.js` route still works as a fallback.

Remember to hit Publish. Base44 code edits do not go live until the app is published.

The one thing that WILL bite you: Base44 apps are single-page apps

Base44's own docs say apps are **client-side rendered (CSR)**. That matters because `whop.track("page")` in the base snippet is a **single top-level call**. It runs once, when the script first executes.

- The first page a visitor lands on is recorded.
- **Every client-side route change after that is invisible**, because the script never re-runs.

So page-view counts will look far too low unless the developer calls `whop.track("page")` again on each route change. In a React router that is one `useEffect` keyed on the current location:

```
// runs on every client-side navigation
useEffect(() => {
  if (window.whop) window.whop.track("page");
}, [location.pathname]);
```

This does not affect revenue. The purchase event is what carries the money and the match keys, and it fires from the order confirmation regardless. The page event affects attribution and audience quality, so it is worth doing, but do not let it hold up the install.

The purchase event is still a developer job

Nothing about Base44 changes step 2. Wherever the app renders the order confirmation, it has to call the purchase event with the **real order id and the real order total**, taken from the order object rather than from anything on the page. Note that Base44 apps can have server functions, and if the store has one that finalizes the order, firing the purchase server side from there is more reliable than firing it in the browser, for the same reason it is on any other platform: it does not depend on the buyer's browser reaching a receipt screen.

Checklist for a Base44 install

1. Code tab, edit `index.html`, paste the step 1 snippet into the `<head>`, with the account id above already filled in. **Publish.**

2. Add the `whop.track("page")` call on route change.
3. Wire the purchase event where the order confirmation is rendered, using the real order id and total, with `event_id` set as described in step 3 so nothing double counts.
4. Verify using the method in "How to verify the install", before sending any traffic.